

The Signal Grid Report

Nakul Kadam

1) Getting image ready for the processing (Section 1 of .m file) :

Image from the folder is loaded.

It is converted to gray scale.

Value-range is shrunk from 0to255 to 0to1.

Why did I do this? :

Since given example pics are for grayscale, the code was written for gray (2D) images.

PSNR, SSIM, colormap require values 0to1.

How did I do the above? :

First double() brings the values to decimals 0 to 255.

Then shrink the range to 0to1 by dividing by 255.

Code snippets :

```
18
19     img_original = imread(filename);
20
21     img_gray = double(img_original);
22
23
24     img_double = img_gray / 255.0;
```

double() makes the values decimal.

*this is the gray image
0to1*

2) Get rid of the Salt&Pepper noise (Section 2 of .m file) :

We slide a square kernel over every pixel and pick the median of the values.

Why did I do this? :

Salt&Pepper noise pixels stand out from their neighbouring pixels (sudden 0(black) or sudden white(255)).

Median of the values kills the spiked value out of the range.

How did I do the above? :

Kernel is created.

A dummy image is used to hover the kernel; which has the OG gray image in the centre and extensions on outer borders and the 4 corners; so that the kernel has no 'empty' pixels.

The kernel is hovered over the OG gray image in the centre (256*256 pixels) and the centre value of the kernel (median) is pasted on new filtered image.

Code snippets:

① kernel created

```
37 KERNEL_SIZE = 3;  
38 half = floor(KERNEL_SIZE / 2); = 1 (for 3x3)  
39 img_filtered = zeros(size(img));
```

② concatenate the dummy image

(had to be written like this as kernel is of variable size)

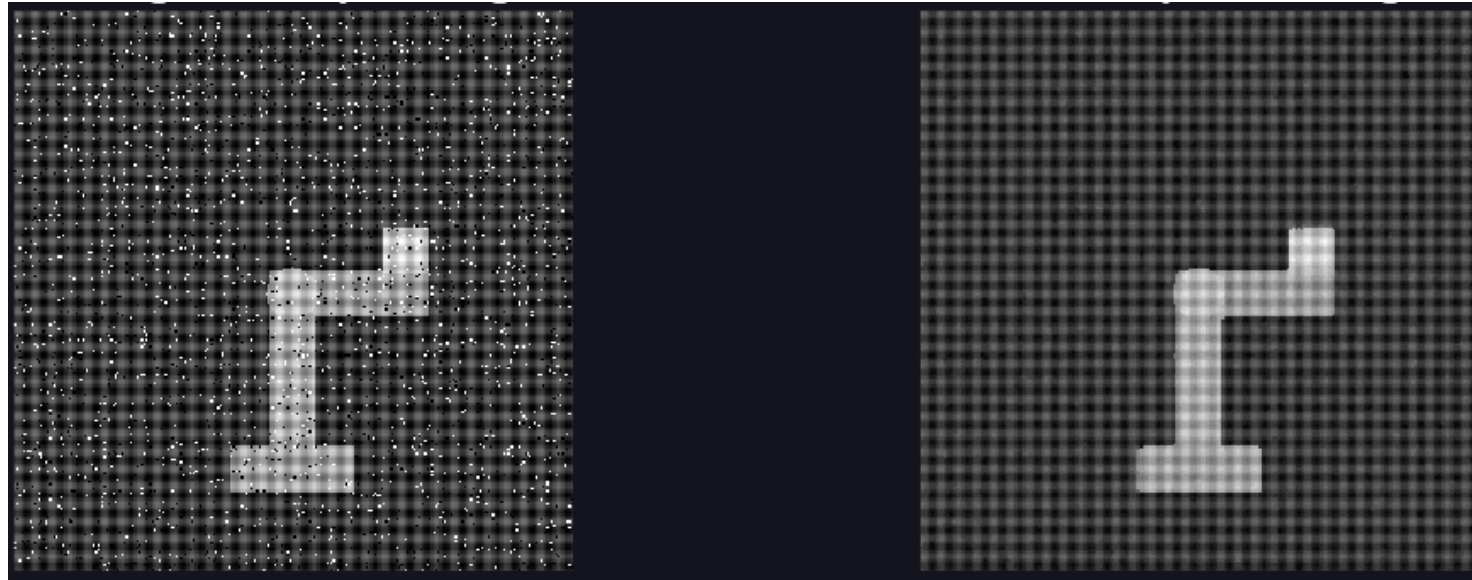
```
% create the dummy image on which the kernel will hover (258x258). for 3x3  
img_padded = [img_double(half:-1:1, half:-1:1),... (1,1) % Top left corner reflected.  
img_double(half:-1:1, :), ... (1,:) % Top edge reflected above itself.  
img_double(half:-1:1, end:-1:end-half+1);... (1,end) % Top right corner reflected.  
  
img_double(:, half:-1:1), ... (:,1) % Left edge reflected to it's left.  
img_double,... % Centre.  
img_double(:, end:-1:end-half+1);... (:,end) % Right edge reflected to it's right.  
  
img_double(end:-1:end-half+1, half:-1:1), ... (end,1) % Bottom Left corner reflected.  
img_double(end:-1:end-half+1, :),... (end,:) % Bottom edge reflected below itself.  
img_double(end:-1:end-half+1, end:-1:end-half+1)];(end,end) % Bottom Right corner reflected.
```

③ Hover the kernel & paste the median value

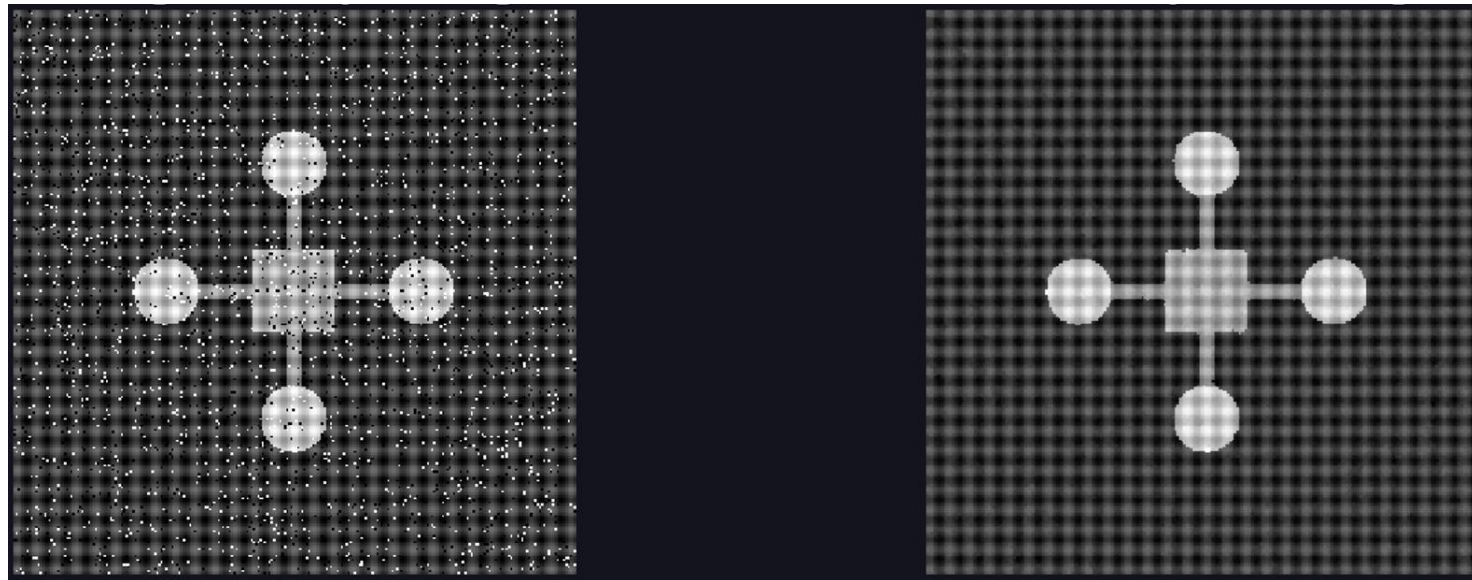
```
53  
54 for r = 1 : rows % Sliding the kernel.  
55 for c = 1 : cols  
56  
57 patch = img_padded(r : r + KERNEL_SIZE - 1, ... % Each 3x3 patch from img_padded is copied as 3x3 array.  
58 c : c + KERNEL_SIZE - 1);  
59  
60 neighbourhood = patch(:); % Each 3x3 patch is converted to a column of 9 values.  
61 sorted_vals = sort(neighbourhood); % The 9 values are arranged in asc order.  
62 median_index = ceil(numel(sorted_vals) / 2); % Index of the median value is picked.  
63 img_filtered(r, c) = sorted_vals(median_index); % The central tile location is assigned with the median value.  
64 end  
65 end
```

Output images of Spatial filtering : Before vs After

Example 1
(cursed_schematic_02)



Example 2
(cursed_schematic_03)



3) Quality metrics (Section 3 of the .m file) :

We use PSNR & SSIM to roughly measure how good the processing work has been performed.

Why did I do this? :

Showing numerical evidence is more convincing than showing montage of prev vs after filtering.

PSNR (Peak Signal to Noise Ratio) is ratio of max signal value to MSE (Mean Square Error) of noise. (The higher the PSNR, the less is the difference betn the peak signal & error).

For our project, PSNR stays ~20dB (indicates noticeable distortion) which is expected as we compare noisy image to the filtered image.

SSIM (Structural Similarity Index Measure) compares the structure (brightness, contrast, etc.) as produces output betn 0 to 1. Closer to 1 indicates highly similar images.

For our project, SSIM stays betn 0.75 to 0.87 which indicates structure is highly preserved.

How did I do the above? :

PSNR, SSIM, MSE have standard formulae. Other tiny variables that were needed to compute this were used as per the rules.

Code snippets :

```
79 mse_val = mean((img_double(:) - img_filtered(:)).^2); } standard formula of MSE
80 if mse_val == 0
81     psnr_val = Inf; } if MSE = 0, means images same. ∴ PSNR → ∞
82 else
83     psnr_val = 10 * log10(1.0 / mse_val); } standard formula for PSNR
84 end
85
86
87 xmu = mean(img_double(:));
88 ymu = mean(img_filtered(:));
89 xsig = std(img_double(:));
90 ysig = std(img_filtered(:));
91 xysig = mean((img_double(:) - xmu) .* (img_filtered(:) - ymu));
92 C1 = (0.01)^2;
93 C2 = (0.03)^2;
94 ssim_val = (2*xmu*ymu + C1) * (2*xysig + C2) / ((xmu^2 + ymu^2 + C1) * (xsig^2 + ysig^2 + C2)); } standard SSIM formula
95
```

standard variables required for SSIM

4) Detecting & getting rid of the periodic grid noise (Section 4,5,6 of .m file) :

Filtered image is converted to frequency domain.

Periodic noise is identified in the Frequency domain. (For a periodic grid noise, the frequency domain looks like peaks along a cross +).

Create a mask according to riden the grid noise but protect the OG structure.

Come back to Spatial domain.

Why do I do this? :

The grid noise is seperable in the Frequency domain rather than Spatial.

Masking out the unwanted frequency will also remove it's part in the Spatial domain.

How did I do the above? :

Convert to 2D Frequency domain using `fft2()` and then shift the DC component (lowest frequency) to the centre using `fftshift()`.

Create a mask of all 1s... and makes 0s along the arms of the cross. (Thickness of the mask's arms can be varied).

A circular patch at the centre (DC component) is retained in the mask. (Radius of mask circle can be varied).

The mask is multiplied to the frequency spectrum and result becomes frequency domain of final filtered image.

The DC centre is put back to it's place using `ifftshift()` and converted back to Spatail domain using `ifft2()`.

Only real part of `ifft2()` is extracted using `real()`. Ideally imaginary part should cancel out as the OG signal is Real; but since computers use Floating Point Arithmetic, tiny unwanted rounding errors of order 10^{-15} may remain.

Code snippets :

① Go to frequency domain

```
109
110     F      =fft2(img_filtered);
111     F_shifted =fftshift(F);
112
113     magnitude_spectrum =log(1 + abs(F_shifted));
114
```

Many trial n errors were done with DC protection radius & mask arm width... this set of combination produced the best output.
Anything more/less either retained the grid more or distorted the DC structure.

② Masking the grid frequency while retaining DC component.

```
121
122     % suppress the frequency cross
123     DC_PROTECT_R    = 16;
124     CROSS_WIDTH     = 2;
125
126     mag = abs(F_shifted);
127
128     cy = floor(rows/2) + 1;
129     cx = floor(cols/2) + 1;
130
131     [X, Y] = meshgrid(1:cols, 1:rows);
132     dc_zone = ( (X - cx).^2 + (Y - cy).^2 ) <= ( DC_PROTECT_R^2 );
133
134     freq_mask = ones(rows, cols);
135
136     freq_mask(:, cx-CROSS_WIDTH : cx+CROSS_WIDTH) = 0;
137
138     freq_mask(cy-CROSS_WIDTH : cy+CROSS_WIDTH, :) = 0;
139
140     freq_mask(dc_zone) = 1;
141
142     n_peaks = sum(freq_mask(:) == 0);
```

% Radius about centre of DC component; the circular region needs to be protected.
% Half-width of the cross.
% Magnitude of the complex numbers in F_shifted.
% Centre coordinates. For 256x256, cx=cy=129.
% meshgrid here returns two 256x256 arrays; X holds column of every position and Y holds row of every position.
% The circle of DC component (to be protected).
% Start the mask with all 1s (keep everything rn, we'll make it 0 when we detect noise).
% Mask out the vertical strip of the noise cross (+).
% Now mask out the remaining horizontal strip of the noise.
% In the process, we masked out the DC component, so we undo it by making the circular region as 1s.
% Count of spikes detected.

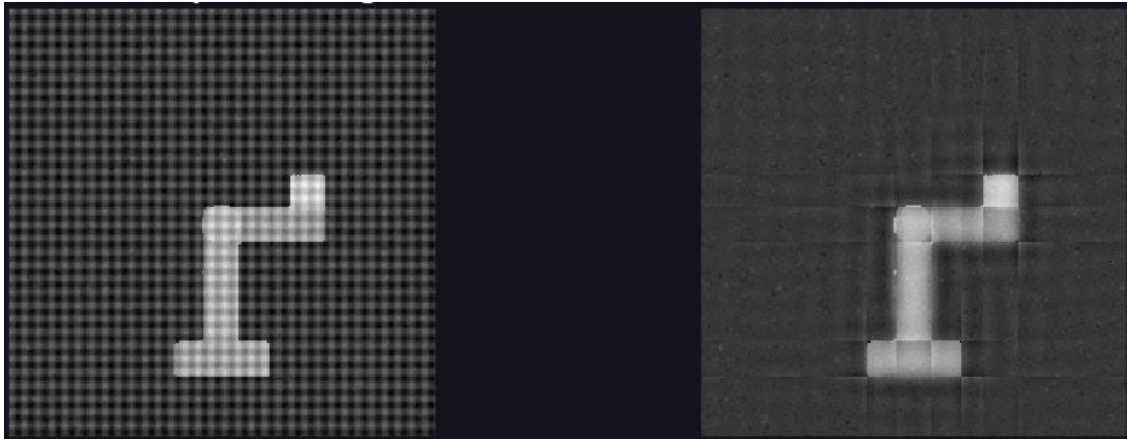
③ Apply mask, shift DC centre, come back to Spatial.

```
150     F_cleaned = F_shifted .* freq_mask;
151     F_ishift =ifftshift(F_cleaned);
152     img_restored =real(ifft2(F_ishift));
153
154     img_restored =max(0, min(1, img_restored));
155
```

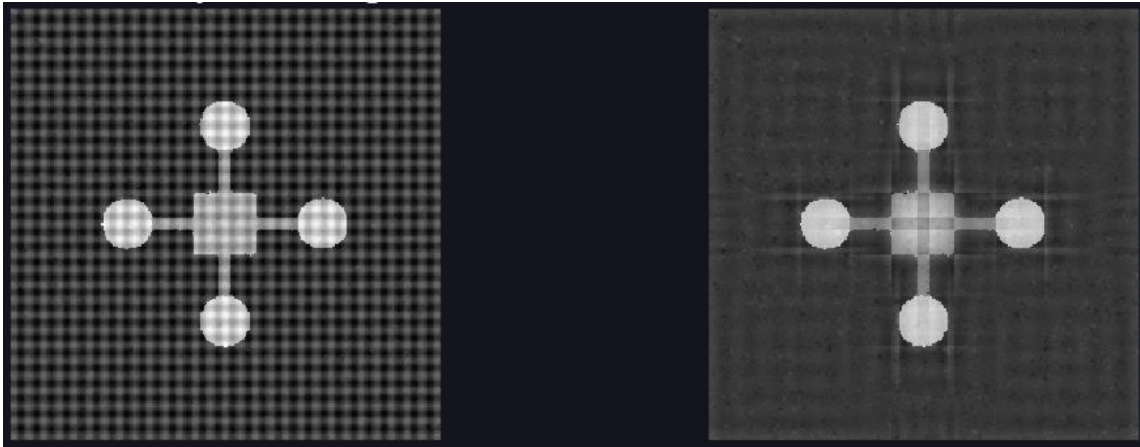
% Multiply the freq spectrum by the mask; pixels with 1 stays, while picels with 0 get eliminated.
% The DC component is shifted back to it's original place.
% Only real parts of the complex numbers is picked out.

Only a safety
measure to avoid getting
values outside 0 to 1

Output images of Frequency filtering : Before vs After

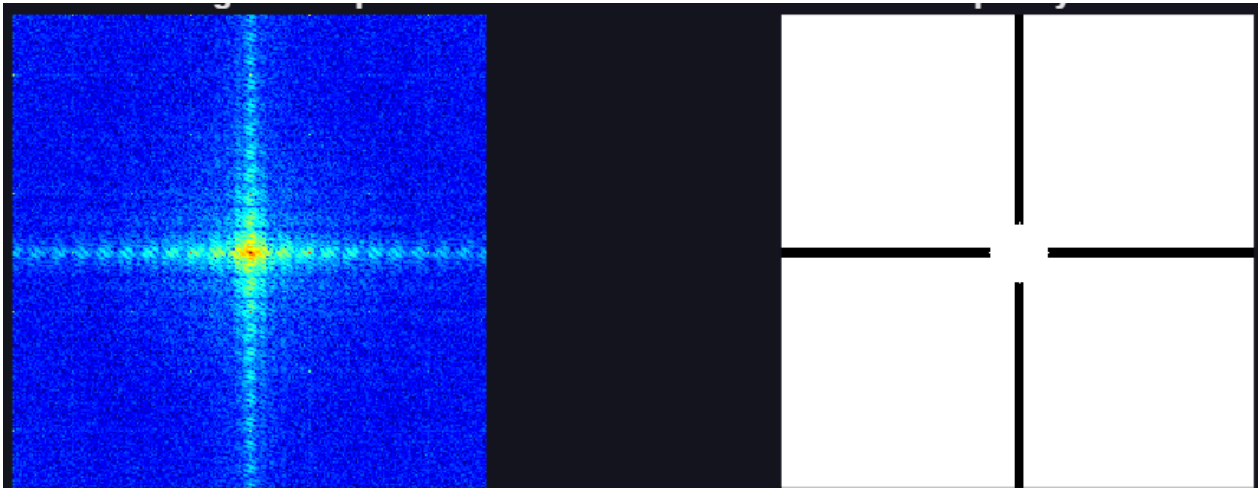


Example 1 (cursed_schematic_02)



Example 2 (cursed_schematic_03)

Output images of Frequency domain and Notch mask :



Example (cursed_schematic_02)

5) Blueprint (Section 7 of the .m file) :

Built a custom 256 color RGB table that maps brightness to blue-shades.

Why did I do this? :

Bonus marks dedo Amaan bhaiyaaa!.

How did I do the above? :

Learnt about LUT (LookUp Table) method; which is commonly used in image processing.

Custom RGB palette was made of 256 blue shades; using `linspace(0,1,256)`.

Corresponding to these values, respective R,G,B values were created as 3 independent row vectors.

These were arranged to form a 256x3 matrix, each row representing a shade of blue (RGB).

The 0to1 range of the gray image is safely converted to 1to256 range and each pixel's value picks it's color from the 256x3 matrix!!

Code snippets :

```

167 n_colors =256;
168 t=linspace(0, 1, n_colors);
169
170 rbp = 0.05*t;
171 gbp = 0.4 * t.^1.5;
172 bbp = 0.3 + 0.7*t;
173
174 blueprint_cmap= [rbp, gbp, bbp];
175 blueprint_cmap= max(0, min(1, blueprint_cmap));
176
177 blueprint_cmap = reshape(blueprint_cmap, [], 3);
178 idx = max(1, min(256, round(img_restored * 255) + 1));
179
180
181
182
183 blueprint_img = reshape(blueprint_cmap(idx(:), :), rows, cols, 3);
184

```

t is 0 0.004 1

rbp is 0 0.0002 0.05

gbp is 0 0.0002 0.4

bbp is 0.3 0.303 1.0

Color for fixed value 1

Color for fixed value 2

again... just for safety even tho we saw that values are both 0 to 1

this part creates the color palette

final blueprint

% linspace creates 1x256 (evenly spaced numbers from 0 to 1).

% Changing the below given coefficients will only result in different shades of blue (may shrink the range of blue as well).

% red part of the color. Stays near 0 for blue.

% green part of the color.

% blue part of the color.

% this creates 1x768.

% min(1,...) ensures maximum is 1;

% capping at 1 was valid, but I didnt want unexpected errors of having <0 so I added max(0,...) so that minimum is 0.

% makes it 256x3. Each row is a RGB color. It gets lighter as we go top to bottom.

% just an intermediate image, it's values will tell the RGB row number it needs. img_restored is 0to1;

*% while we need 1to256. *255 and round() to get 0to255; add 1 to make it 1to256.*

% if the round() step exceeds 256, min(256,...) handles it.

% max(0,...) handles if negative number appears. (idk how it's possible, but looking at max(1,...) I thought it'll be safe).

% idx(:) flattens 256x256 to 65536x1. blueprint_cmap(...) fetches the RGB color as per row's number.

% reshape() folds back → 256x256x3.

6) Outputs (Section 8,9 of the .m file) :

Display the desired outputs and save the files (saving can be commented out if not wanted).

How did I do the above? :

A figure was created and it's attributes were decided.

Codes for each subplot were written.

Saved images in the folder being used:

1. task_blueprint (The final processed image blue-version) and 2. task_result (The entire panel)

Code snippets :

①

```
191 figure('Name', 'Full Restoration pipeline', ...
192       'NumberTitle', 'off', ...
193       'Units', 'normalized', ...
194       'OuterPosition', [0 0 1 1], ...
195       'Color', [0.08 0.08 0.12]);
196
197
198 ts={'FontSize', 13, 'FontWeight', 'bold', 'Color', [0.9 0.9 0.9]};
199 xs={'FontSize', 10, 'Color', [0.7 0.7 0.7]};
200
201
202 subplot(2, 3, 1);
203 imshow(img_double, [0,1]);
204 title('1. Original corrupted image', ts{:});
205 xlabel('Salt & pepper + Grid noise', xs{:});
206
207
208 subplot(2, 3, 2)
209 imshow(img_filtered, [0 1]);
210 title('2. After Spatial filtering', ts{:});
211 xlabel(sprintf('PSNR : %.2f dB | SSIM : %.4f', psnr_val, ssim_val), xs{:});
212
213
```

Attributes of panel 1
provide settings

② figure (panel display)

```
214 subplot(2, 3, 4);
215 imshow(magnitude_spectrum, []);
216 colormap(gca, jet);
217 title('3. 2D FFT Magnitude Spectrum', ts{:});
218 xlabel('Centre = DC | Bright spikes = grid noise', xs{:});
219
220
221 subplot(2, 3, 5);
222 imshow(freq_mask, [0 1]);
223 title('4. Frequency Mask', ts{:});
224 xlabel(sprintf('White = kept | Black = zeroed | %d peaks masked', n_peaks), xs{:});
225
226
227 subplot(2, 3, 6);
228 imshow(blueprint_img);
229
230 title('5. Final Restored image (Blueprint)', ts{:});
231 xlabel('Frequency noise eliminated', xs{:});
232
233
234 sgtitle('Image Restoration Pipeline | Task1:Spatial + Task2:Frequency', 'FontSize', 16, 'FontWeight', 'bold', 'Color', [0.0 0.9 1.0]);
235
```

% FFT magnitude spectrum
% Frequency mask.
% The blueprint image.

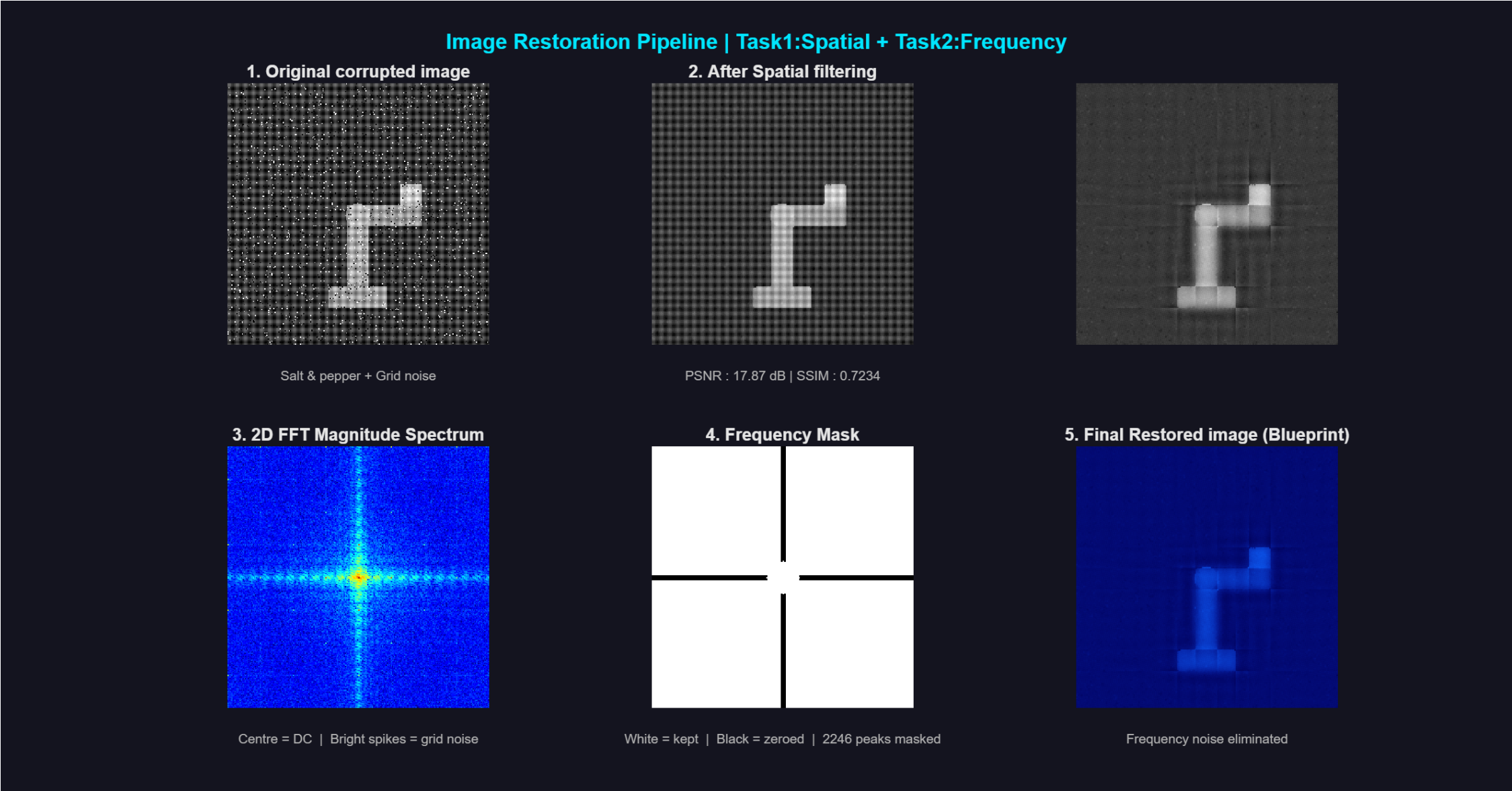
③ Saving the 2 results

```
imwrite(blueprint_img, 'task_blueprint.png');

saveas(gcf, 'task_result.png');

fprintf('\nOutputs saved:\n');
fprintf(' task_blueprint.png - blueprint colourised version\n');
fprintf(' task_result.png - full 5-panel figure\n');
fprintf('\nPipeline complete.\n');
```

Output image of Final view panel :



Example (task_result for cursed_schematic_02)